

GD RDM 프로젝트를 하나의 CLI로 작업을 하다가 그 창을 닫고 다시 몇시간 있다가 다시 cmd로 창을 열어 이어서 작업을 하면 이게 하나의 세션이야 아님 다른 세션이 생긴거야?

**다른 세션이 생긴 거야.**

터미널 창을 닫으면 세션이 종료되고, 새로 cmd를 열어서 claude를 실행하면 새 세션 ID가 생성된다.

---

## 정확한 흐름

첫 번째 작업

cmd 창 열기 → claude 실행 → 세션 A (JSONL 파일 생성됨)

... 작업 중 ...

cmd 창 닫기 → 세션 A 종료 (JSONL은 디스크에 보존)

몇 시간 후

cmd 창 새로 열기 → claude 실행 → 세션 B (새 JSONL 파일 생성)

← 세션 A의 컨텍스트는 없음, 백지 상태

---

## 이전 작업을 이어가려면

창을 새로 열었을 때 이전 세션을 이어가는 방법은 세 가지야.

# 방법 1: 가장 최근 세션을 바로 이어감 (제일 편함)

claude --continue

# 방법 2: 목록 보고 골라서 재개

claude --resume

# 방법 3: 특정 세션 ID 지정

claude --resume <session-id>

---

## 실무 팁

GD RDM 프로젝트처럼 **머칠에 걸쳐 작업하는 경우**, --continue를 습관화하는 게 좋아. 그러면 이전 대화 컨텍스트가 그대로 살아나서 CLAUDE.md를 다시 설명하지 않아도 되거든.

단, 세션이 너무 길어지면 컨텍스트 윈도우가 꽉 찰 수 있으니 Feature 단계가 완전히 마무리됐을 땐 새 세션으로 시작하고 CLAUDE.md에 진행 상황을 기록해두는 게 깔끔해.

---

claude --continue으로 이어서 하면 그게 하나의 세션이 계속 이어지는거야?

맞아, **하나의 세션이 계속 이어지는 거야.**

--continue는 새 세션을 만드는 게 아니라, 종료됐던 이전 세션의 JSONL 파일을 불러와서 그 컨텍스트 위에서 대화를 재개하는 방식이야. 세션 ID도 그대로 유지돼.

---

## 정확한 흐름

첫 번째 작업

claude 실행 → 세션 A 시작

... 작업 ...

cmd 창 닫기 → 세션 A 일시 중단 (JSONL 저장)

몇 시간 후

claude --continue → 세션 A 재개 (같은 세션 ID)

... 이전 대화 컨텍스트 그대로 살아있음 ...

cmd 창 닫기 → 세션 A 다시 중단

또 몇 시간 후

claude --continue → 세션 A 또 재개

사람이 잠자고 일어나서 다시 일하는 것처럼, 세션은 디스크에서 "잠든" 상태였다가 --continue로 깨어나는 개념이야.

---

## 단, 주의할 점 하나

--continue는 **가장 최근 세션**을 불러오는 거라서, 만약 그 사이에 실수로 claude를 그냥 실행(인자 없이)했다면 새 세션 B가 "가장 최근"이 되어버려. 그럴 땐 claude --resume으로 목록을 보고 세션 A를 직접 골라서 재개해야 해.

GD RDM 프로젝트처럼 장기 작업은 **항상 --continue로 열고, 실수로 그냥 claude를 쳤다면 바로 나와서 --resume으로 올바른 세션을 찾는 습관**을 들이면 돼.

---

claude --resume 을 입력하면 세션 목록이 나오는거 같은데 그 목록명은 뭘로 저장되는거야?

claude --resume을 입력하면 나오는 목록은 별도의 "이름"이 있는 게 아니라, 아래 정보들을 조합해서 표시해 줍니다.

---

## 목록에 표시되는 정보

- 날짜/시간 — 세션이 마지막으로 업데이트된 시각
- 첫 번째 사용자 메시지의 앞부분 — 그 세션에서 제일 처음 입력한 메시지가 미리보기 텍스트로 표시됨

예를 들면 이런 식으로 보여요.

> 2026-06-20 14:32 "rdm\_merge.py에서 542컬럼 병합 로직 작성해줘..."

2026-06-19 10:11 "Feature 3 분석 모듈 시작하자..."

2026-06-18 09:05 "안녕, GD RDM 프로젝트 세팅부터..."

---

## 실제 저장 파일명 (세션 ID)

내부적으로는 ~/.claude/projects/<프로젝트경로>/ 아래에 **UUID 형태의 JSONL**

파일로 저장됩니다.

~/claude/projects/C:\wclaude\projects\GDW

|—— a3f8d2c1-4b5e-7f9a-...jsonl   ← 세션 A  
|—— b7c4e1d0-2a3f-8c1b-...jsonl   ← 세션 B  
|—— ...

이 UUID가 곧 세션 ID이며, `claude --resume <session-id>`로 특정 세션을 지정할 때 이 값을 사용하게 됩니다.

정리

사용자가 지정하는 세션 이름은 따로 없고, **"첫 메시지 미리보기 + 날짜"** 조합으로 구분하는 방식입니다. 그래서 GD RDM 프로젝트처럼 장기 작업을 할 때는 첫 메시지를 "Feature 2 병합 로직 작업 시작" 처럼 명확하게 입력해 두면 나중에 목록에서 찾기가 훨씬 편합니다.

schema-first 패턴이 뭐야?

Schema-First 패턴이란?

Schema-First 패턴은 데이터나 시스템을 설계할 때 **"스키마(구조 정의)"**를 가장 먼저 확정하고, 그 이후의 모든 작업(코드 작성, 데이터 생성, 처리 로직 등)을 그 스키마를 기준으로 진행하는 개발 방식입니다.

핵심 개념

일반적인 개발 순서와 비교하면 이해하기 쉽습니다.

| 구분        | 일반적인 방식 (Code-First / Data-First) | Schema-First        |
|-----------|-----------------------------------|---------------------|
| 시작점       | 코드나 실제 데이터부터 작성                   | 데이터 구조(스키마) 정의부터 시작 |
| 스키마 결정 시점 | 나중에 코드/데이터에서 역추론                  | 처음에 명시적으로 확정        |
| 변경 시 영향   | 코드 곳곳에 흩어진 가정들이                   | 스키마 한 곳만 수정하면       |

| 구분 | 일반적인 방식 (Code-First / Data-First) | Schema-First |
|----|-----------------------------------|--------------|
|    | 깨짐                                | 전체에 전파       |

왜 사용하나요?

### 1. 단일 진실 공급원(Single Source of Truth)

- 컬럼명, 타입, 제약조건 등이 스키마 파일 하나에 정의되고, 모든 코드(생성/병합/분석/대시보드)가 이를 참조합니다.

### 2. 불일치 방지

- 예를 들어 컬럼이 542개인 대규모 데이터를 다룰 때, 각 단계(생성 → merge → 분석 → 웹 대시보드)마다 컬럼명이나 타입을 따로 하드코딩하면 어딘가에서 반드시 어긋납니다. 스키마를 먼저 정의해두면 이런 문제를 원천 차단할 수 있습니다.

### 3. 검증(Validation)의 기준점

- 실제 데이터가 스키마와 맞는지 자동으로 검증할 수 있습니다. (예: schema-validator 서브에이전트)

### 4. 문서화 효과

- 스키마 자체가 "이 데이터가 무엇인지"를 설명하는 문서 역할을 합니다.

## GD RDM 프로젝트에 적용하면

최상길님의 GD RDM 프로젝트 구조로 예를 들면:

### 1. 스키마 정의 (542개 컬럼의 이름, 타입, 제약조건을 먼저 확정)

↓

### 2. rdm\_data\_generator.py → 스키마를 참조해서 데이터 생성

↓

### 3. rdm\_merge.py → 스키마 기준으로 병합 시 컬럼 정합성 검증

↓

### 4. rdm\_analysis.py → 스키마에 정의된 타입에 맞춰 분석 로직 적용

↓

5. 웹 대시보드 → 스키마 메타데이터를 기반으로 UI 자동 생성 가능

이렇게 하면 CLAUDE.md에 스키마를 명시해두고, 각 단계의 코드가 "스키마를 어기지 않는지"를 기준으로 작업하게 되어 대규모 CSV 처리에서 발생하기 쉬운 컬럼 누락/타입 불일치 문제를 크게 줄일 수 있습니다.

---

혹시 GD RDM 프로젝트의 스키마 정의 파일 구조(예: JSON Schema, Pydantic 모델, YAML 등)를 어떻게 잡을지 구체적으로 설계해 드릴까요?

---

Schema-First 패턴 특성상 Claude는 실제 데이터를 직접 보지 못하고 메타데이터만 보고 코드를 작성합니다. 라는게 무슨 뜻이야?

좋은 질문입니다. 이 말은 Claude(또는 개발자)가 코드를 작성할 때 **실제 데이터 값(row 하나하나)**을 보지 않고, 데이터의 "**구조에 대한 설명**"만 보고 판단한다는 뜻입니다.

**좀 더 풀어서 설명하면**

일반적으로 데이터를 다룰 때 코드를 짜는 방식은 두 가지로 나뉩니다.

### 1) 데이터를 직접 보고 짜는 방식 (Data-Driven)

실제 CSV 파일을 열어서

"아, IMEI 컬럼에 이런 값들이 들어있구나"

"온도 컬럼은 -20~85 사이 숫자구나"

→ 이걸 보고 코드 작성

### 2) 메타데이터만 보고 짜는 방식 (Schema-First)

실제 데이터는 안 보고, 아래 같은 "설명서"만 봄:

```
{  
  "column": "temp_sensor_01",  
  "type": "float",  
  "unit": "celsius",
```

```
"range": [-20, 85],  
"nullable": false  
}
```

→ 이 스펙만 보고 코드 작성

## 왜 이런 차이가 생기나요?

GD RDM 프로젝트처럼 **회사 보안 규정상 실제 Galaxy 진단 데이터를 외부 AI 서비스(Claude)에 노출할 수 없는 환경**에서는, Claude가 실제 542개 컬럼의 실데이터 값을 볼 방법이 없습니다.

그래서:

- ❌ Claude가 실제 CSV를 열어보고 "이 컬럼엔 이런 패턴의 값이 있네" 하고 추론 → **불가능**
- ✅ 컬럼명, 타입, 제약조건, 예시 값(마스킹되거나 합성된) 등이 담긴 **스키마 문서만 전달받아서** 그걸 기준으로 코드 작성 → **가능**

## 이게 실무에 주는 영향

| 항목    | 영향                                                                                             |
|-------|------------------------------------------------------------------------------------------------|
| 장점    | 민감 데이터 유출 위험 없이 AI 활용 가능. 스키마만 정확하면 코드 품질은 동일하게 유지됨                                            |
| 위험 요소 | 스키마 문서가 실제 데이터와 어긋나 있으면(스키마엔 없는데 실제로 있는 이상치, 결측치 패턴 등) Claude가 짠 코드가 실제 데이터에서 예상 밖의 동작을 할 수 있음 |
| 대응 방법 | schema-validator 서브에이전트처럼, <b>실제 데이터가 스키마를 준수하는지 검증하는 단계</b> 를 반드시 별도로 두어야 함                   |

즉, Claude는 "이 데이터는 이런 규칙을 따를 것이다"라는 **\*\*약속(계약, contract)\*\***을 믿고 코드를 짜는 것이고, 그 약속이 맞는지 확인하는 건 별도의 검증 로직(실제 데이터를 다루는 회사 내부 환경)의 몫이 되는 구조입니다.

데이터를 직접 보고 짜는 방식 (Data-Driven)으로 코드를 짰다면 만약 그 데이터가 엄청 큰 CSV 파일이라면 Claude code는 실제 그 파일을 read\_csv()

같은 걸로 읽는거야? 만약 그렇다면 토큰의 소모가 엄청 커지는 거 아냐?

좋은 포인트를 짚으셨습니다. 이 부분은 헷갈리기 쉬운데, `"""`코드가 데이터를 읽는 것`"""`과 `"""Claude(모델)가 데이터를 읽는 것"""`을 구분해서 봐야 합니다.

**핵심: 실행 환경과 Claude의 컨텍스트는 분리되어 있습니다**

Claude Code는 로컬(또는 샌드박스) 환경에서 **Python/Bash 코드를 실행**할 수 있는 도구입니다. 이때 두 가지 경로가 있습니다.

**경로 1) 코드가 데이터를 읽고 처리 (토큰 소모 거의 없음)**

```
import pandas as pd
```

```
df = pd.read_csv("huge_file.csv") # 이 데이터는 Python 프로세스의 메모리에 로드됨
```

```
result = df.groupby("model").mean()
```

```
result.to_csv("summary.csv")
```

- 이 `read_csv()`는 **Python 인터프리터 메모리**에서 실행됩니다.
- Claude(모델)의 컨텍스트 윈도우에는 `"""`이 코드 자체와 실행 결과(출력 로그)`"""`만 들어갑니다.
- 즉, 542개 컬럼 × 수백만 행의 실제 값은 Claude의 "눈"에 전혀 노출되지 않습니다.

**경로 2) Claude가 파일 내용을 직접 "본다" (토큰 폭발)**

```
cat huge_file.csv
```

또는 view 툴로 CSV 파일 전체를 열람하는 경우

- 이걸 파일 내용이 **텍스트로 그대로 컨텍스트에 삽입**되는 방식입니다.
- 수백만 행짜리 CSV라면 순식간에 컨텍스트 한도를 초과하거나 비용이 급증합니다.

**그래서 실무에서는 이렇게 합니다**

| 단계    | 방식                                                                                                   | 토큰 영향 |
|-------|------------------------------------------------------------------------------------------------------|-------|
| 구조 파악 | <code>df.head()</code> , <code>df.dtypes</code> , <code>df.columns</code> 등<br><b>샘플/요약만</b> 출력해서 확인 | 매우 적음 |



| 단계    | 방식                                         | 토큰 영향               |
|-------|--------------------------------------------|---------------------|
| 실제 처리 | pandas/polars 등으로 <b>코드가 전체 데이터를 처리</b>    | 없음 (실행 환경 메모리에서 처리) |
| 결과 확인 | 처리 결과의 <b>요약 통계, 에러 메시지, 일부 샘플만</b> 확인     | 적음                  |
| 검증    | assert, 스키마 검증 로직의 <b>pass/fail 결과만</b> 확인 | 매우 적음               |

즉, Claude Code가 read\_csv()를 실행하는 것 자체는 문제가 없습니다. 문제는 **그 결과를 얼마나 많이 Claude의 컨텍스트로 되돌려 보내느냐**입니다.

### Schema-First가 여기서도 유리한 이유

GD RDM처럼 542개 컬럼짜리 대용량 CSV를 다룰 때, Schema-First 접근은 보안 문제뿐 아니라 **토큰 효율성** 측면에서도 이득입니다.

- Claude가 **\*\*스키마 문서(수백 줄)\*\***만 보고 코드를 작성 → 컨텍스트 매우 가벼움
- 만약 Data-Driven으로 접근했다면, "이 컬럼에 어떤 값들이 들어있는지 실제로 확인해봐"라는 요청이 반복될 때마다 df.head(50), df["col"].unique() 같은 샘플 출력을 계속 컨텍스트에 쌓게 되어 비효율적입니다.

### 정리

- **Claude Code가 코드를 실행해서 큰 CSV를 처리하는 것:** 문제없음 (실행 환경 메모리 사용, 토큰 소모 없음)
- **Claude가 그 데이터를 "읽고 이해하려는" 목적으로 대량의 내용을 컨텍스트에 끌어오는 것:** 토큰 소모 큼
- 실무에서는 **샘플링 + 요약 출력**으로 최소한의 정보만 Claude에게 전달하고, 실제 대량 처리는 코드가 백그라운드에서 수행하도록 설계합니다.

---

### 컨텍스트는 뭐야?

Claude Code에서 **\*\*컨텍스트(Context)\*\***는 현재 세션에서 Claude가 "알고 있는" 모든 정보의 총합입니다.

---

## 구체적으로 무엇이 포함되나

- 사용자가 입력한 메시지 전체
- Claude가 응답한 내용 전체
- 도구 호출 기록 (파일 읽기, bash 실행 결과 등)
- CLAUDE.md 같은 시스템 프롬프트

이 모든 것이 하나의 **긴 텍스트 덩어리**로 Claude에게 전달됩니다.

---

## 컨텍스트 윈도우란

Claude가 한 번에 처리할 수 있는 컨텍스트의 최대 크기를 **컨텍스트 윈도우**라고 합니다. 토큰(token) 단위로 측정되며, 세션이 길어질수록 누적된 대화가 이 한계에 가까워집니다.

세션 시작

↓ 메시지 주고받을수록

컨텍스트 점점 쌓임

↓ 한계에 도달하면

오래된 내용이 밀려나거나 → /compact로 압축 필요

---

## 세션과의 관계

| 개념   | 설명                                |
|------|-----------------------------------|
| 세션   | 대화의 생명주기 단위 (JSONL 파일로 디스크에 저장)   |
| 컨텍스트 | 세션 안에서 현재 Claude가 메모리에 올려두고 있는 내용 |

세션은 디스크에 영구 저장되지만, 컨텍스트는 현재 활성화된 메모리 공간입니다. --continue로 세션을 재개하면 이전 JSONL을 읽어서 컨텍스트를 복원하는 방식입니다.

---

## GD RDM 프로젝트 관점에서

542컬럼 스키마 설명, CSV 파일 내용, 코드 등이 모두 컨텍스트에 쌓이기 때문에 작업이 길어지면 /compact로 압축하거나, 핵심 내용은 CLAUDE.md에 기록해서 새 세션에서도 빠르게 컨텍스트를 복원할 수 있도록 관리하는 것이 중요합니다.

---

Claude.ai에서 이렇게 내가 하나 질문을 하고 그 답변을 받고 하는것 하나 하나는 뭐라고 하는거야?

각각을 "턴(Turn)" 이라고 합니다.

---

### 정확한 용어 정리

| 용어               | 설명                           |
|------------------|------------------------------|
| 턴(Turn)          | 사용자 메시지 1개 + Claude 응답 1개의 쌍 |
| 메시지(Message)     | 사용자 또는 Claude가 보낸 개별 발화 하나   |
| 대화(Conversation) | 여러 턴이 쌓인 전체 흐름               |

---

### 예시로 보면

[턴 1]

사용자: "세션이 뭐야?"

Claude: "세션은 ..."

[턴 2]

사용자: "컨텍스트는 뭐야?"

Claude: "컨텍스트는 ..."

[턴 3]

사용자: "이걸 워드로 저장해줘"

Claude: "..."

이렇게 지금 이 대화도 여러 개의 턴으로 구성되어 있습니다.

---

Claude API 관점에서도 동일하게 턴이라는 단위를 사용하며, 컨텍스트 윈도우 한계도 "몇 턴까지 기억할 수 있는가"로 체감하게 됩니다.

---

### Claude code의 컨텍스트 윈도우 한계는 얼마야?

최신 정보를 검색해서 확인해 드릴게요.모델별로 정리해 드릴게요.

#### Claude Code 컨텍스트 윈도우 한계

현재 기준(2026년 6월)으로 Claude Fable 5, Opus 4.8, Sonnet 4.6은 모두 **1,000,000 토큰(1M 토큰)** 컨텍스트 윈도우를 가지고 있으며, Haiku 4.5는 **\*\*200,000 토큰(200K 토큰)\*\***입니다.

---

#### 1M 토큰이 실제로 어느 정도냐면

1M 토큰은 약 75만 단어 분량의 영문 텍스트로, 장편 소설 10권 분량이나 5만 줄짜리 Python 프로젝트 전체가 들어가는 크기입니다.

---

#### 그래도 생각보다 빨리 찰 수 있습니다

코딩 세션에서는 수십 개의 파일을 읽고, 빌드를 실행하고, 여러 차례 리팩토링을 거치다 보면 2시간짜리 세션만으로도 현실적으로 10만~30만 토큰을 소비할 수 있습니다.

---

#### 한계에 가까워지면 어떻게 되나

컨텍스트가 가득 차면 하드 에러가 나기 전에 먼저 품질 저하 증상이 나타납니다. Claude가 이전에 결정한 내용을 잊어버리거나, 서로 모순된 응답을 하거나, 앞서 논의한 함수명이나 파일명을 잘못 말하는 현상이 발생합니다.

---

#### GD RDM 프로젝트 관점에서

Claude Code는 컨텍스트 윈도우가 얼마나 찼는지 실시간으로 표시해 줍니다. 새

작업으로 전환할 때는 /clear, 작업 도중 계속 이어가야 할 때는 /compact를 사용하는 것이 기본 원칙이며, Claude Code는 한계에 가까워지면 자동으로 압축(auto-compact)도 해줍니다.

542컬럼 스키마 + CSV 파일 내용 + 코드가 동시에 컨텍스트에 쌓이는 GD RDM 프로젝트는 세션이 길어질수록 토큰 소비가 빠를 수 있으니, Feature 단계가 마무리될 때마다 /clear로 정리하는 습관이 중요합니다.

---

/clear 로 정리하면 내가 이어서 계속 작업할때 위의 내용을 다시 알려줘야 하는거 아니야?

네, 맞습니다. /clear를 하면 컨텍스트가 초기화되기 때문에 Claude는 이전 대화 내용을 전혀 모르는 상태가 됩니다.

그래서 **CLAUDE.md**가 핵심 해결책입니다.

---

## CLAUDE.md의 역할

CLAUDE.md는 세션이 시작될 때마다 **자동으로 컨텍스트에 로드**됩니다. /clear를 하거나 새 세션을 시작해도 CLAUDE.md는 항상 살아있습니다.

/clear 실행

↓

대화 히스토리는 사라짐

↓

하지만 새 세션 시작 시 CLAUDE.md 자동 로드

↓

Claude가 프로젝트 맥락을 즉시 파악

---

## GD RDM 프로젝트 기준으로 CLAUDE.md에 담아둘 내용

- 프로젝트 개요 (542컬럼 스키마, Schema-First 원칙)
- 현재까지 완료된 Feature 단계
- 주요 파일 구조 (rdm\_data\_generator.py, rdm\_merge.py 등)

- 진행 중인 작업과 다음 할 일
- 주의사항 및 컨벤션

## 정리하면

|              | 대화 히스토리 | CLAUDE.md |
|--------------|---------|-----------|
| /clear 후     | 사라짐     | 살아있음      |
| 새 세션 시작 후    | 없음      | 자동 로드됨    |
| --continue 후 | 복원됨     | 자동 로드됨    |

그래서 CLAUDE.md를 잘 관리해두면 /clear나 새 세션을 시작해도 "처음부터 다시 설명"하는 수고 없이 바로 작업을 이어갈 수 있습니다. CLAUDE.md가 사실상 프로젝트의 **영구 기억** 역할을 하는 셈입니다.

그렇다면 왜 굳이 /clear를 해야 하는거야? 그냥 다 찰때까지 계속 쓰다 auto compact되면 마칠때까지 그냥 기다리면 되는거야 아냐?

맞는 말씀입니다. 사실 꼭 /clear를 해야 하는 건 아닙니다.

다만 /clear를 권장하는 이유가 있습니다.

## Auto-compact에만 맡겼을 때의 문제

Auto-compact는 컨텍스트가 꽉 찰 때 자동으로 오래된 대화를 **요약·압축**합니다. 그런데 이 과정에서 이런 문제가 생길 수 있습니다.

- **중요한 결정 사항이 요약되면서 디테일이 날아갈 수 있음**
- **압축 타이밍을 내가 통제 못함** — 작업 한가운데서 갑자기 발생할 수 있음
- **비용 낭비** — 필요 없는 과거 대화(이미 끝난 Feature 1 내용 등)가 계속 토큰을 잡아먹음

## 그래서 언제 /clear가 진짜 유용하냐면

"Feature 2 완료 → Feature 3 시작" 처럼 **명확히 작업 단위가 바뀔 때**입니다.

Feature 2의 디버깅 기록, 시행착오 대화 같은 건 Feature 3에선 전혀 필요 없는데, auto-compact 없이 그냥 두면 그게 계속 토큰을 차지합니다.

현실적인 결론

| 상황                       | 권장 방법           |
|--------------------------|-----------------|
| 같은 작업을 계속 이어갈 때          | 그냥 두거나 /compact |
| 새 Feature·새 작업으로 전환할 때   | /clear          |
| 컨텍스트가 거의 찼는데 작업이 안 끝났을 때 | /compact        |

CLAUDE.md만 잘 관리되어 있다면, /clear 후에도 맥락 복원이 빠르기 때문에 두려워할 필요가 없습니다. 하지만 귀찮으시면 auto-compact에 맡겨도 크게 문제는 없습니다. 다만 비용과 품질 면에서 살짝 손해를 볼 수 있다는 정도입니다.

개인적으로 프로젝트의 크기가 커지면서 중요한 내용을 놓치지 않기 위해서 1. dialog.txt:전체 대화 내용을 그대로 저장, 2.session\_log.txt:세션별 주요 사항저장, CLAUDE.md:이건 Claude code가 알아서 저장하고 프로젝트 폴더의 변경사항은 Git으로 추적/저장하고 있어. 그런데 매번 주요 작업 후 이것들을 일일이 업데이트 해달라고 하는 것도 귀찮고 그렇다고 안하면 중요한 내용이 누락될 수 있는 위험성도 있어. 그런데 매번 이런 작업을 요청하면 그에 따른 토큰 소모도 분명히 유의미하게 있을 거 같아. 이런 상황에서 어떻게 하면 프로젝트의 변경 사항들을 효율적으로 관리할 수 있을지 검토하고 개선안을 제안해 주

좋은 고민이십니다. 현재 방식의 문제점부터 짚고 개선안을 제안드릴게요.

현재 방식의 문제점

| 파일              | 문제                           |
|-----------------|------------------------------|
| dialog.txt      | 전체 대화를 수동 저장 → 토큰 낭비 + 누락 위험 |
| session_log.txt | 매번 수동 요청 필요 → 귀찮음 + 일관성 없음   |
| CLAUDE.md       | Claude가 알아서 업데이트하지만 타이밍이 불규칙 |

| 파일  | 문제                            |
|-----|-------------------------------|
| Git | 코드 변경은 잘 잡히지만 "왜 바꿨는지" 맥락은 없음 |

핵심 문제는 "**수동 요청**"에 의존하고 있다는 점입니다.

---

## 개선안: Hook 기반 자동화

Claude Code의 **Hook** 기능을 활용하면 수동 요청 없이 자동으로 기록할 수 있습니다.

### Hook이란?

특정 이벤트(파일 저장, 세션 종료, 도구 호출 등)가 발생할 때 **자동으로 스크립트를 실행**하는 기능입니다. 컨텍스트 윈도우 바깥에서 실행되므로 토큰 소모가 없습니다.

---

## 구체적인 구조 제안

### 1) dialog.txt → 폐기 권장

세션 JSONL 파일이 이미 전체 대화를 저장하고 있습니다. 별도로 dialog.txt를 유지하는 건 중복입니다.

~/.claude/projects/GD/<session-id>.jsonl

→ 이게 곧 dialog.txt입니다

필요할 때 JSONL을 파싱해서 읽으면 되므로 dialog.txt는 없애도 됩니다.

---

### 2) session\_log.txt → Hook으로 자동화

세션 종료 시(PostSession Hook) 자동으로 핵심 사항을 추출해서 기록하도록 합니다.

```
// .claude/settings.json
```

```
{
```

```
  "hooks": {
```

```
    "PostSession": [
```



```
{
  "matcher": "",
  "hooks": [{
    "type": "command",
    "command": "python scripts/save_session_log.py"
  }]
}
]
```

# scripts/save\_session\_log.py

# 세션 종료 시 자동 실행

# - 오늘 날짜/세션 ID

# - 수정된 파일 목록 (Git diff로 추출)

# - 마지막 N개 메시지 요약

# → session\_log.txt에 자동 append

---

### 3) CLAUDE.md → Hook + Git commit 메시지로 보완

CLAUDE.md는 Claude가 업데이트하되, **Git commit** 시점을 **CLAUDE.md 업데이트 트리거**로 활용합니다.

작업 완료

→ git commit

→ PostToolUse Hook 실행

→ CLAUDE.md의 "현재 진행 상황" 섹션 자동 업데이트

---

### 4) Git commit 메시지를 구조화

commit 메시지 자체를 session\_log 대응으로 활용합니다.

feat(Feature3): 분석 모듈 이상값 탐지 로직 추가

- 542컬럼 중 수치형 컬럼 대상 Z-score 기반 탐지
- 임계값 설정 가능하도록 config.yaml에 파라미터 추가
- 다음 작업: 웹 대시보드 연동

이렇게 하면 git log --oneline만 봐도 세션별 진행 상황이 한눈에 들어옵니다.

---

### 최종 제안 구조

작업 중

└─ Claude가 파일 수정 → Git이 자동 추적

작업 마무리 시 (하루 1회 정도)

- └─ git commit (구조화된 메시지)
  - └─ Hook → CLAUDE.md "진행 상황" 섹션 자동 갱신
  - └─ Hook → session\_log.txt 자동 append

필요할 때만

└─ JSONL 파싱으로 전체 대화 열람 (dialog.txt 불필요)

---

### 토큰 관점에서의 이득

| 작업            | 기존                     | 개선 후                |
|---------------|------------------------|---------------------|
| dialog.txt 저장 | 매번 Claude에게 요청 → 토큰 소모 | JSONL 그대로 활용 → 0 토큰 |
| session_log   | 매번 Claude에게 요청 → 토큰    | Hook 스크립트 → 0 토큰    |

| 작업                | 기존             | 개선 후                   |
|-------------------|----------------|------------------------|
| 업데이트              | 소모             |                        |
| CLAUDE.md<br>업데이트 | 수시로 요청 → 토큰 소모 | commit 시 Hook →<br>최소화 |

## 결론

지금 당장 적용할 수 있는 우선순위는 이렇습니다.

1. **즉시** — dialog.txt 폐기, JSONL로 대체
2. **단기** — git commit 메시지 구조화 (템플릿 만들어 두기)
3. **중기** — PostSession Hook으로 session\_log.txt 자동화
4. **중기** — CLAUDE.md 자동 갱신 Hook 연결

Hook 스크립트 실제 코드가 필요하시면 말씀해 주세요. GD RDM 프로젝트 구조에 맞게 바로 작성해 드릴 수 있습니다.

## JSONL이 뭐야?

**\*\*JSONL(JSON Lines)\*\***은 JSON 객체를 한 줄에 하나씩 저장하는 텍스트 파일 형식입니다.

## 일반 JSON vs JSONL 비교

**일반 JSON** — 전체가 하나의 덩어리

```
[
  {"role": "user", "content": "안녕"},
  {"role": "assistant", "content": "안녕하세요!"}
]
```

**JSONL** — 한 줄에 JSON 하나씩

```
{"role": "user", "content": "안녕"}
{"role": "assistant", "content": "안녕하세요!"}
```

---

## Claude Code가 JSONL을 쓰는 이유

대화가 진행되면서 **한 줄씩 계속 append(추가)** 하면 되기 때문입니다.

일반 JSON은 새 내용을 추가할 때마다 파일 전체를 다시 써야 하지만, JSONL은 맨 아래에 한 줄만 추가하면 되므로 대화 기록처럼 **계속 누적되는 데이터**에 매우 적합합니다.

---

## 실제 Claude Code의 세션 JSONL 구조

```
{"type": "user", "content": "세션이 뭐야?", "timestamp": "2026-06-20T14:32:01Z"}
```

```
{"type": "assistant", "content": "세션은 하나의 터미널...", "timestamp": "2026-06-20T14:32:05Z"}
```

```
{"type": "tool_use", "tool": "read_file", "path": "rdm_merge.py", "timestamp": "2026-06-20T14:33:10Z"}
```

```
{"type": "user", "content": "컨텍스트는 뭐야?", "timestamp": "2026-06-20T14:35:22Z"}
```

턴마다 한 줄씩 쌓이기 때문에 나중에 파싱하면 전체 대화를 순서대로 복원할 수 있습니다.

---

## 정리

| 항목                | 내용                    |
|-------------------|-----------------------|
| 확장자               | .jsonl                |
| 구조                | 한 줄 = JSON 객체 하나      |
| 장점                | 빠른 append, 대용량 처리에 유리 |
| Claude Code에서의 역할 | 세션 전체 대화 기록 저장소       |

앞서 말씀드린 대로 이 JSONL 파일이 사실상 dialog.txt와 동일한 역할을 하고 있기 때문에, 별도로 dialog.txt를 관리할 필요가 없다는 것입니다.

---

이 파일이 모든 대화를 다 저장하고 있어?

네, 저장하고 있습니다. 다만 몇 가지 중요한 특성이 있습니다.

---

### 저장되는 것

- 사용자가 입력한 모든 메시지
  - Claude의 모든 응답
  - 도구 호출 기록 (파일 읽기, bash 실행, 결과값 등)
  - 각 메시지의 타임스탬프
  - 토큰 사용량 등 메타데이터
- 

### 주의할 점

**1) /clear를 하면 JSONL도 분리됩니다** /clear 이후 대화는 같은 세션 JSONL에 이어 쓰이는 게 아니라, 내부적으로 새 블록으로 구분될 수 있습니다.

**2) /compact를 하면 압축된 요약으로 대체됩니다** 압축 전 원본 대화가 JSONL에서 요약문으로 치환될 수 있어서, 압축 이전의 세부 내용이 날아갈 수 있습니다.

**3) Bash 명령어 결과는 일부 잘릴 수 있습니다** 출력이 너무 긴 경우 전체가 아닌 일부만 저장될 수 있습니다.

---

### 결론

**일반적인 대화 흐름은 전부 저장됩니다.** 다만 /compact를 자주 쓰신다면 그 시점 이전 내용의 디테일이 손실될 수 있으므로, 중요한 내용은 session\_log.txt나 CLAUDE.md에 별도로 기록해두는 것이 안전합니다.

---

compact 시 압축된다는 이게 바로 문제야 내가 dialog.txt를 이용하는 이유야 맞습니다. 정확히 핵심을 짚으셨습니다.

---

### 문제의 본질

/compact 실행

↓

JSONL의 이전 대화 내용이 요약문으로 치환됨

↓

"A 방법을 시도했다가 실패해서 B로 바꿨다"는 맥락이

"B 방법을 사용함" 한 줄로 압축될 수 있음

↓

나중에 "왜 B를 썼지?" 를 추적할 수 없게 됨

특히 GD RDM처럼 장기 프로젝트에서는 **시행착오의 맥락**이 매우 중요한데, 이게 compact로 날아가면 나중에 같은 실수를 반복하거나 왜 그런 결정을 했는지 알 수 없게 됩니다.

---

그래서 **dialog.txt** 방식이 맞습니다

|           | JSONL        | dialog.txt   |
|-----------|--------------|--------------|
| compact 후 | 이전 내용 손실     | 전부 보존        |
| 형식        | 기계가 읽기 좋은 형식 | 사람이 읽기 좋은 형식 |
| 검색        | 파싱 필요        | 텍스트 검색 바로 가능 |
| 누적        | 자동           | 수동 요청 필요     |

---

그렇다면 **현재 방식의 진짜 문제는**

dialog.txt 자체가 나쁜 게 아니라 **"매번 수동으로 요청해야 한다"** 는 점입니다.

앞서 제안드린 **Hook 자동화**가 여기서 진가를 발휘합니다. 매 턴이 끝날 때마다 Hook으로 대화 내용을 자동으로 dialog.txt에 append하면, 수동 요청 없이도 한 글자도 빠짐없이 누적 저장이 가능합니다.

관심 있으시면 GD RDM 프로젝트에 맞는 자동 저장 Hook 스크립트를 바로 작성해 드릴 수 있습니다.

---

작성해 줘봐

## GD RDM 프로젝트 — Hook 자동화 스크립트

### 파일 구성

```
<프로젝트 루트>/                                ← C:\Claude\projects\GDW
|
|—— .claude/
|   |—— settings.json                            ← Hook 연결 설정
|   |—— scripts/
|       |—— save_dialog.py                       ← 매 턴마다 dialog.txt 자동 저장
|       |—— save_session_log.py                 ← 세션 종료 시 session_log.txt 자동 저장
|—— dialog.txt                                  ← 자동 생성됨 (전체 대화 누락 없이 기록)
|—— session_log.txt                             ← 자동 생성됨 (세션별 요약 기록)
```

### 설치 방법

1. 아래 파일들을 GD 프로젝트 루트에 복사합니다.

.claude/settings.json

scripts/save\_dialog.py

scripts/save\_session\_log.py

2. Python 3이 설치되어 있는지 확인합니다.

```
python --version
```

# Python 3.x.x 이상이면 OK

3. Claude Code를 프로젝트 루트에서 실행하면 Hook이 자동 활성화됩니다.

```
cd C:\Claude\projects\GD
```

```
claude
```

### 동작 방식

#### dialog.txt (save\_dialog.py)

- **트리거:** 매 턴에서 도구 호출(bash, 파일 읽기/쓰기 등)이 완료될 때마다 실행
- **기록 내용:** 사용자 메시지, 도구 호출 결과, Claude 응답 전체

- /compact를 해도 dialog.txt에는 원본이 그대로 보존됨 ← 핵심 장점
- 토큰 소모 없음 (컨텍스트 윈도우 바깥에서 실행)

#### session\_log.txt (save\_session\_log.py)

- 트리거: 세션 종료 시 (터미널 창 닫기, exit 등) 1회 실행
- 기록 내용: 세션 ID, 총 턴 수, 첫 메시지(주제), 마지막 응답(결과), 도구 사용 현황, Git 변경 파일 목록
- 세션별 "무엇을 했는가"를 빠르게 파악하는 용도

#### 기록 예시

##### dialog.txt

```
=====
=====
```

GD RDM 프로젝트 — 대화 전체 기록 (dialog.txt)

시작 시각: 2026-06-20 14:32:01

```
=====
=====
```

[2026-06-20 14:32:01] [세션: a3f8d2c1...]

 USER

rdm\_merge.py에서 542컬럼 병합 로직 작성해줘

 TOOL CALL — read\_file

파일: rdm\_merge.py

--- 내용 ---

...

 ASSISTANT



rdm\_merge.py를 분석했습니다. 현재 구조는 ...

-----

## session\_log.txt

=====

세션 종료: 2026-06-20 18:00:00

세션 ID: a3f8d2c1-4b5e-7f9a-...

총 턴 수: 24

=====

### ▶ 세션 시작 메시지 (주제)

rdm\_merge.py에서 542컬럼 병합 로직 작성해줘

### ▶ 세션 마지막 응답 (최종 결과)

rdm\_merge.py의 병합 로직이 완성되었습니다. 총 542컬럼을 ...

### ▶ 사용된 도구

- bash: 18회
- read\_file: 12회
- write\_file: 6회

### ▶ Git 변경 사항 (세션 종료 시점)

rdm\_merge.py | 127 ++++++

config.yaml | 8 +-  
+-----+

---

## 주의사항

- dialog.txt와 session\_log.txt는 .gitignore에 추가하는 것을 권장합니다.  
(용량이 커질 수 있고, 민감한 대화 내용이 포함될 수 있음)
- Bash 명령어로 직접 실행한 스크립트 결과(예: python rdm\_merge.py 출력)는 Hook이 stdout을 그대로 기록하므로 매우 길어질 수 있습니다.  
MAX\_SUMMARY\_LEN 설정값으로 조절 가능합니다.
- Hook 자체 오류는 scripts/hook\_error.log에 기록되며, dialog.txt나 세션 동작에는 영향을 주지 않습니다.